

Smart Campus Room Booking

Author: Mohammad Taosif Ikram

Department: CSE

Daffodil International University

Introduction

Universities provide students and staff with shared spaces and equipment such as study rooms, meeting rooms, and common rooms, along with resources like whiteboards, webcams, and microphones. Without a proper system, it can be difficult to know which rooms and resources are free, and clashes or confusion can easily happen.

This report contains the design and testing of a Smart Campus Room Booking & Resource Manager, a terminal-based Python application that allows users to add rooms and resources, view them, search for them, and update their booking status.

The system was built to satisfy the assessment requirements. The application uses a user-choice interface controlled by a while loop and if/elif statements to process user choices. Separate functions are used for each feature. Two Python dictionaries, one for rooms and one for resources, are used as the main data structures. Input validation is also used throughout the program to make sure that IDs stay unique, capacity is always a positive number, and status values are limited to “Available” or “Booked”.

The remainder of this report analyzes the problem, describes the system design including two flowcharts, explains the Python implementation, and presents a set of test cases that show that the system behaves correctly for both normal and abnormal inputs.

Problem Analysis

The task is to create a simple digital system to manage university study rooms and shared resources. The system allows users to add new rooms and resources, view all recorded information, search for rooms by type or resources by category, and change their status between “Available” and “Booked”.

If a user tries to update a room or resource that does not exist, the system displays a clear error message instead of crashing or doing nothing.

To solve the problem, we identified these tasks:

- A. Store room ID, type, capacity, and status.
- B. Store resource ID, name, category, condition, and status.
- C. Stop users from adding duplicate IDs.
- D. Make sure room capacity is a positive number.
- E. Allow only “Available” or “Booked” as status.
- F. Show all rooms and resources clearly.
- G. Search rooms by type and resources by category.

- H. Update the status of rooms or resources and show an error message if the ID is not found.
- I. Keep the program running until the user chooses to exit.

Rooms and resources are stored separately because they contain different types of information, but both need similar operations such as adding, viewing, searching, and updating. For this reason, Python dictionaries are used to store the data. One dictionary is used for rooms and another for resources. Each room or resource has a unique ID, which is used as the key in the dictionary. The other details, such as type, capacity, category, condition, and status, are stored in a nested dictionary. This makes the data easy to organise, find, and update. It also makes it simple to check whether an ID already exists and prevent duplicate entries.

System Design

Data Structure

The system is built using two Python dictionaries, which are used to store information about rooms and resources. This follows the requirements of the assignment and keeps the data organised.

The first dictionary is called rooms. It stores information about all the study rooms in the system. The Room ID is used as the key, for example, “R201”. The value of each key is another dictionary that stores the room's type, capacity, and status.

The second dictionary is called resources. It stores information about shared resources such as microphones, webcams, or whiteboards. The Resource ID is used as the key, for example, “MIC-01”. The value is another dictionary containing the resource's name, category, condition, and status.

For example, after adding one room and one resource, the dictionaries may look like this:

```
rooms = {  
    "R201": {  
        "type": "Study Room",  
        "capacity": 4,  
        "status": "Available"  
    }  
}
```

```
resources = {  
    "MIC-01": {  
        "name": "Wireless Mic",  
        "category": "Audio",  
        "condition": "Good",  
        "status": "Available"  
    }  
}
```

This type of structure is useful because all the information related to a room or resource is kept together. The unique ID also makes it easy to find a particular room or resource. For example, the program can check whether “R201” already exists before adding a new room. It can also use the ID to find a room or resource and change its status from “Available” to “Booked” or from “Booked” to “Available”.

Using dictionaries also makes the program easier to manage because there is no need to search through a long list of records. The ID can be used directly to access the required information. Therefore, dictionaries are a suitable data structure for this simple room and resource management system.

Menu-Driven Structure

The program uses a menu-driven structure so that users can easily select the operation they want to perform. The main part of the program is controlled by a `main()` function, which contains a `while True` loop. This loop allows the program to continue running and showing the menu until the user decides to exit.

At the beginning of each loop, the program displays the available options to the user. The user then enters a number to select an operation. The program uses `if/elif` statements to check the selected option and call the correct function.

For example, when the user wants to add a room, the program can call `add_room()`, and when the user wants to add a resource, the program can call `add_resource()`. Similarly, if the user wants to view all the stored information, they can use the `display()` function. If the user wants to find a specific room or resource, they can use the `search_room()` and `search_resource()` functions. These functions help the user quickly find the required information from the system.

Therefore, the main menu connects all the separate functions and allows the user to control the system from one place.

The menu includes seven options. Options 1–6 are used for adding, displaying, searching, and updating rooms and resources. When the user selects option 7, the program breaks out of the while True loop and displays an exit message before ending.

The program also checks for invalid menu choices. If the user enters something that is not a valid option from 1 to 7, the final else statement displays an “Invalid choice” message. The program does not crash in this situation. Instead, it returns to the beginning of the loop and displays the menu again. This makes the system easier and safer for the user to operate.

Flowchart 1 — Main Menu Logic

Figure 1 shows the main menu process. The program displays the menu, takes the user's choice, and performs the selected action. This process continues until the user selects option 7 to exit.

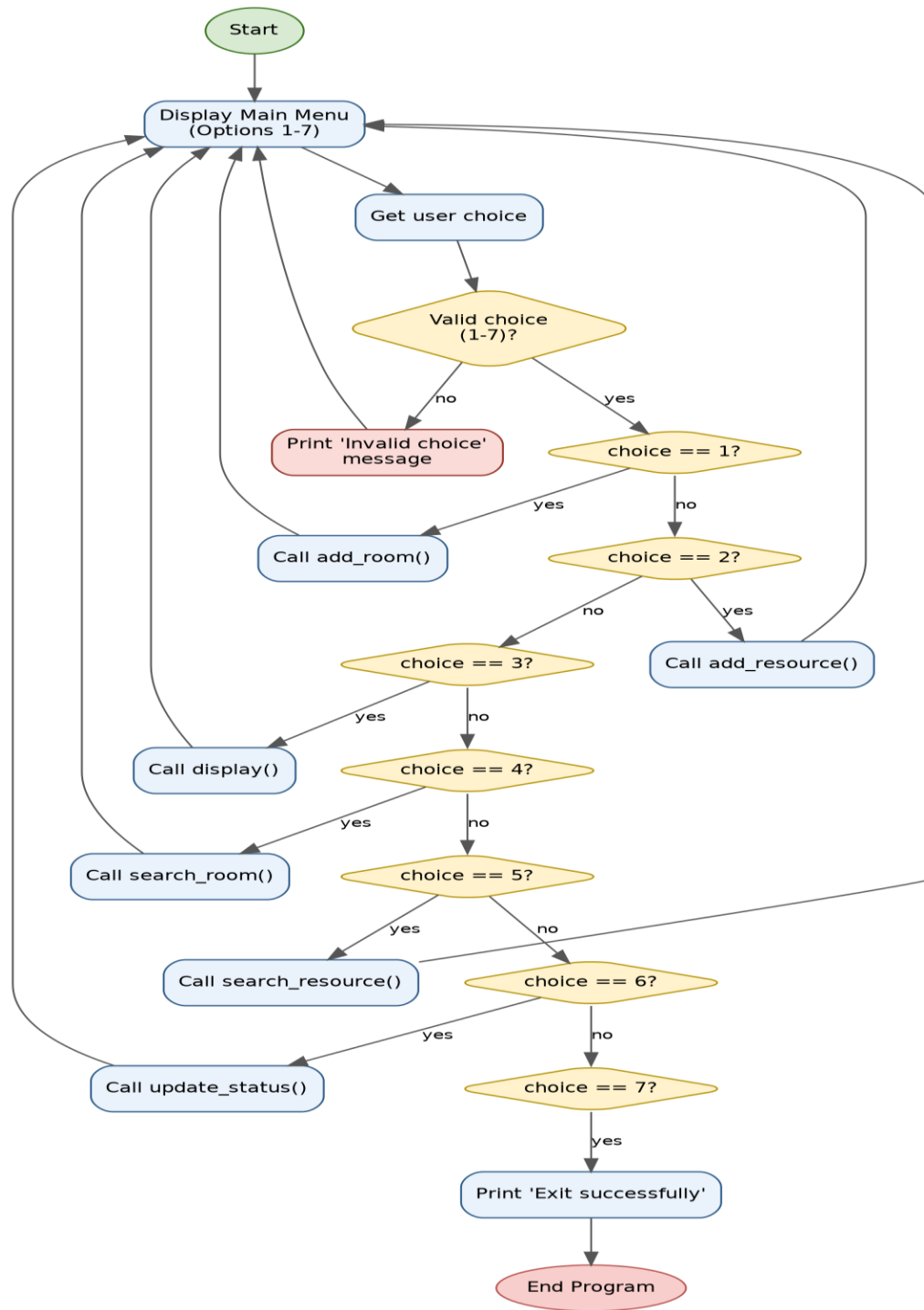


Figure 1: Main Menu Logic Flowchart

3.4 Flowchart 2 — Update Booking Status Function

Figure 2 shows the steps used in the `update_status()` function. **First**, the user chooses whether they want to update a **room** or a **resource**. Then, the program asks for the ID and checks whether it **exists** in the system.

If the ID does not exist, the program shows a clear **error message** and does not make any changes. If the ID exists, the program asks the user to enter a new status. The program then checks whether the status is **“Available”** or **“Booked”**.

If the user enters an invalid status, an error message is shown, and the status is not updated. If the status is valid, the program updates the status of the selected room or resource.

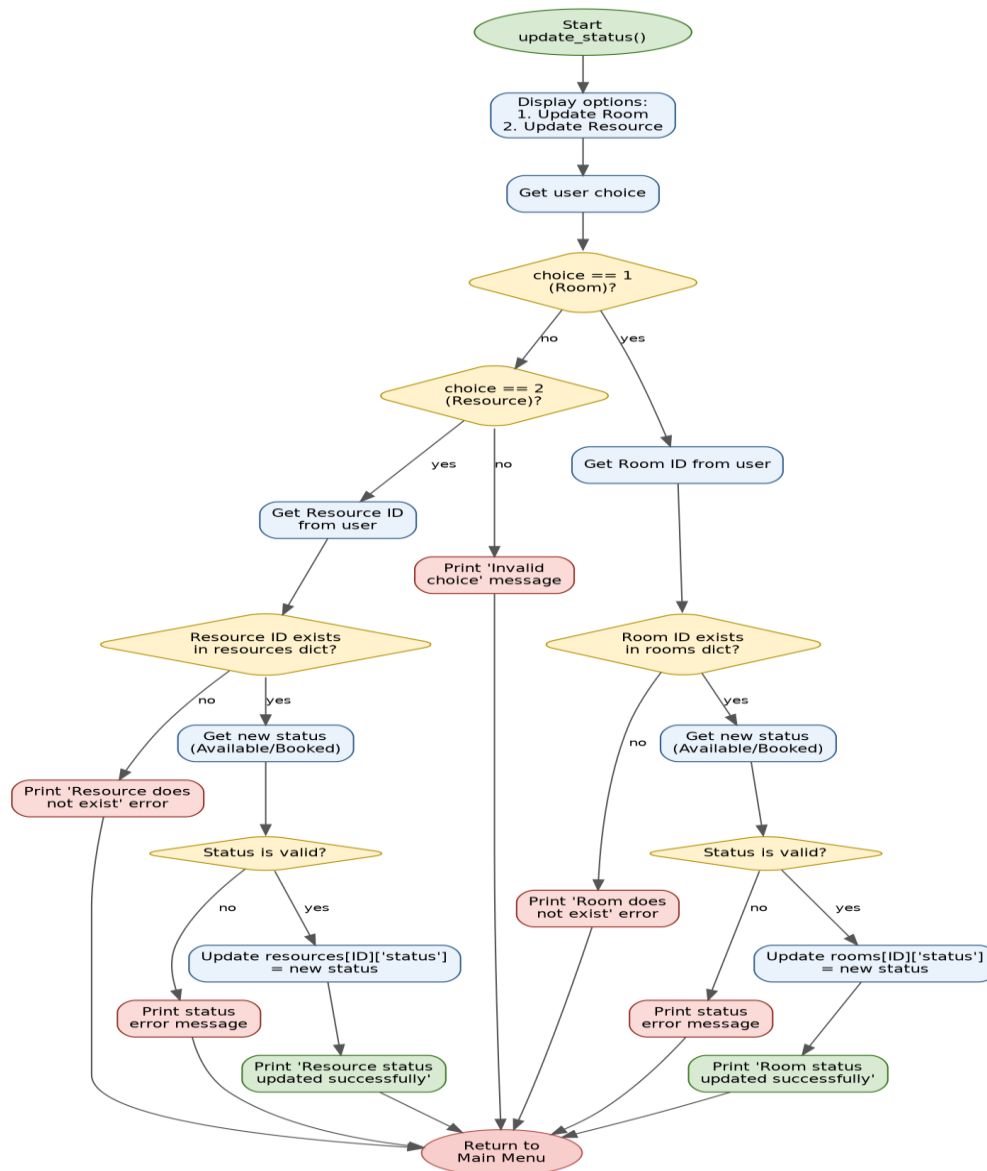


Figure 2: Update Booking Status Function Flowchart

Python Implementation

The program was implemented in a single Python file. Each feature of the menu has its own function, which makes the code easier to understand and organised.

1. Adding Rooms and Resources

The `add_room()` and `add_resource()` functions follow a similar process. First, the program checks whether the ID entered by the user already exists in the relevant dictionary by using the `in` operator. If the ID already exists, an error message is shown and the function stops. This helps to prevent duplicate IDs.

For rooms, the capacity is checked using `isdigit()` and checked to make sure it is greater than zero. This prevents invalid inputs such as "abc" or negative numbers from being stored. Both functions also check that the status is either "Available" or "Booked" before adding the new room or resource to the dictionary.

2. Displaying All Rooms and Resources

The `display()` function shows all the rooms and resources stored in the dictionaries. It goes through the rooms dictionary and displays each room's ID, type, capacity, and status. It then does the same for the resources, showing the ID, name, category, condition, and status.

If there are no rooms or resources stored, the program shows a "No rooms found" or "No resources found" message. This makes the output easier for the user to understand.

3. Searching

The `search_room()` function allows the user to search for a room by its type. It checks each room in the dictionary and compares its type with the user's search input. Both values are converted to lowercase, so the search is not case-sensitive.

Similarly, the `search_resource()` function allows the user to search for resources by category. If a matching room or resource is found, its information is displayed. If nothing matches the search, the program shows a clear "No room found" or "No resource found" message instead of showing a blank result.

4. Updating Booking Status

The `update_status()` function allows the user to change the booking status of a room or resource. First, the user chooses whether they want to update a room or a resource. The program then asks for the relevant ID and checks whether it exists in the correct dictionary.

If the ID does not exist, an error message is shown and no changes are made. If the ID exists, the user is asked to enter a new status. The program checks that the status is either “Available” or “Booked”. If it is valid, the status is updated in the dictionary.

5. Input Validation Summary

The program uses different types of input validation to prevent incorrect data:

- A. Unique IDs:** The in operator is used to check if an ID already exists before adding a room or resource.
- B. Positive capacity:** isdigit() is used to check that the capacity is a number and greater than zero.
- C. Allowed status values:** The program only accepts “Available” or “Booked” as valid status values.
- D. Non-existent IDs:** update_status() checks whether the ID exists before making any changes. If it is not found, an error message is displayed.

The complete source code of the program is provided in Appendix A at the end of this report.

Testing

The system was tested manually by running the program in the terminal and trying different menu options with different inputs. Six test cases are given below. There are **normal cases** with valid IDs and valid status values, and the other three are **abnormal cases** with duplicate IDs, invalid values, and non-existent IDs. Each test case includes the input, expected output, actual output, and the test result to show whether the test passed or failed.

No	Type	Input	Expected Output	Actual Output	Pass/Fail
1	Normal	Add Room → ID: "R101", Type: "Study Room", Capacity: "4", Status: "Available"	Room R101 added successfully and stored in the rooms dictionary.	Room R101 added successfully.	Pass
2	Normal	Add Resource → ID: "MIC-01", Name: "Wireless Mic", Category: "Audio", Condition: "Good", Status: "Available"	Resource MIC-01 added successfully and stored in the resources dictionary.	Resource MIC-01 added successfully.	Pass
3	Normal	Update Status → Room ID: "R101", New Status: "Booked"	Room R101's status is changed to "Booked" and a success message is shown.	Room status updated successfully.	Pass
4	Abnormal	Add Room → ID: "R101" (already exists)	System rejects the entry and shows "Room already exists" without overwriting the existing room.	Invalid room ID. Room already exists.	Pass
5	Abnormal	Add Room → Capacity: "-5"	System rejects the entry because capacity is not a positive number.	Capacity must be a positive number. Try again!	Pass
6	Abnormal	Update Status → Room ID: "R999" (does not exist)	System shows an error message and does not update anything.	The room does not exist.	Pass

All six test cases produced the expected results. Normal cases show that valid data is accepted and stored correctly. The abnormal cases show that the validation rules in Section 4.5 correctly prevent duplicate IDs, invalid capacities, and non-existent IDs from being processed. In each case, the program gives the user a clear error message instead of crashing or giving an unexpected result.

Conclusion

This project successfully developed a working terminal-based **Smart Campus Room Booking & Resource Manager** that meets the main requirements of the assignment.

The system uses two dictionaries to store rooms and resources, a while loop with if/elif statements to control the menu, and separate functions for adding, displaying, searching, and updating information. Input validation is also included to make sure that IDs are unique, capacities are positive, and only **“Available”** or **“Booked”** are accepted as valid status values. The program also shows a clear error message when a user tries to update a room or resource that does not exist.

The testing in Section 5 shows that the system works correctly with both normal and abnormal inputs. In the future, the system could be improved by saving data to a file so that information is not lost when the program closes. It could also support bookings based on specific dates and times instead of only using **“Available”** and **“Booked”** status. Another possible improvement would be to create a graphical or web-based interface instead of using a terminal menu.

Appendix A: Python Source Code

The complete Python source code for the Smart Campus Room Booking & Resource Manager is listed below.

```

# Two dictionaries
rooms = {}
resources = {}

# Add a room
def add_room():
    print("\nAdd Room")
    room_id = input("Enter room ID: ")

# Check room exists
    if room_id in rooms:
        print("Invalid room ID. Room already exists.")
        return
    room_type = input("Enter room type: ")
    capacity = input("Enter capacity: ")

    if not capacity.isdigit() or int(capacity) <= 0:
        print("Capacity must be a positive number. Try again!")
        return
    capacity = int(capacity)
    status = input("Enter status (Available or Booked): ")
    if status not in ["Available", "Booked"]:
        print("Try again! Status must be Available or Booked.")
        return

# Store room Id in the rooms dictionary as the key
    rooms[room_id] = {
        "type": room_type,
        "capacity": capacity,
        "status": status
    }
    print("room added!")

# Add a resource
def add_resource():
    print("\nAdd Resource")

```

```

resource_id = input("Enter resource ID: ")

# Check resource ID to prevent duplication
if resource_id in resources:
    print("Invalid resource ID. Resource already exists.")
    return

name = input("Enter resource name: ")
category = input("Enter category: ")
condition = input("Enter condition: ")

status = input("Enter status (Available or Booked): ")

if status not in ["Available", "Booked"]:
    print("Try again! Status must be Available or Booked.")
    return

resources[resource_id] = {
    "name": name,
    "category": category,
    "condition": condition,
    "status": status
}
print("resource added!")

# Display All Rooms and Resources

def display():
    print("\nAll Rooms")
# Check the rooms dictionary contains any rooms
if len(rooms) == 0:
    print("No rooms found.")
else:
    for room_id in rooms:
        room = rooms[room_id]
        print("\nRoom ID:", room_id)
        print("Type:", room["type"])
        print("Capacity:", room["capacity"])

```

```
        print("Status:", room["status"])
print("\nAll Resources")

if len(resources)==0:
    print("No resources found.")
else:
    for resource_id in resources:
        resource = resources[resource_id]

        print("\nResource ID:", resource_id)
        print("Name:", resource["name"])
        print("Category:", resource["category"])
        print("Condition:", resource["condition"])
        print("Status:", resource["status"])
```

Searching room by type

```
def search_room():
    print("\nSearch Room by Type")
    search_type = input("Enter room type: ")

    found = False
    # Check each room for a matching in the rooms dictionary
    for room_id in rooms:
        room = rooms[room_id]

        if room["type"].lower() == search_type.lower():
            print("\nRoom ID:", room_id)
            print("Type:", room["type"])
            print("Capacity:", room["capacity"])
            print("Status:", room["status"])
            found = True

    if found == False:
        print("No room found. Change the type and try again!")
```

Search resource by category

```

def search_resource():
    print("\nSearch Resource by Category")
    search_category = input("Enter Resource Category: ")
    found = False
    # Check each resource for a matching in the resources dictionary
    for resource_id in resources:
        resource = resources[resource_id]
    # Compare stored resource category with user's search input
    if resource["category"].lower() == search_category.lower():
        print("\nResource ID:", resource_id)
        print("Name:", resource["name"])
        print("Category:", resource["category"])
        print("Condition:", resource["condition"])
        print("Status:", resource["status"])

        found = True

    if found == False:
        print("No resource found.")

# Update Booking Status

def update_status():
    print("\nUpdate Booking Status")
    print("1. Update Room")
    print("2. Update Resource")

    choice = input("Enter your choice: ")

    if choice == "1":
        room_id = input("Enter room ID: ")
    # Check the id exists in the rooms dictionary
    if room_id not in rooms:
        print("The room does not exist.")
        return

    new_status = input(
        "Enter new status (Available/Booked): "

```

```

    )

    if new_status not in ["Available", "Booked"]:
        print("Try again! Status must be Available or Booked.")
        return

# Update room's status
    room = rooms[room_id]
    room["status"] = new_status
    print("Room status updated successfully!")

elif choice == "2":
    resource_id = input("Enter Resource ID: ")
# Check the resource ID exists in the resources dictionary
    if resource_id not in resources:
        print("Error: Resource does not exist.")
        return

    new_status = input(
        "Enter new status (Available/Booked): "
    )

    if new_status not in ["Available", "Booked"]:
        print("Error: Status must be Available or Booked.")
        return

    resource = resources[resource_id]
    resource["status"] = new_status

    print("Resource status updated successfully!")

else:
    print("Invalid choice.")

# Main Menu

def main():
# Keep displaying the menu untill the user choose to exit
    while True:

```

```
print("\nSmart Campus Room Booking Manager")
print("1. Add Room")
print("2. Add Resource")
print("3. Display All")
print("4. Search Room by Type")
print("5. Search Resource by Category")
print("6. Update Booking Status")
print("7. Exit")
```

```
choice = input("Enter your choice (1-7): ")
```

```
if choice == "1":
    add_room()
```

```
elif choice == "2":
    add_resource()
```

```
elif choice == "3":
    display()
```

```
elif choice == "4":
    search_room()
```

```
elif choice == "5":
    search_resource()
```

```
elif choice == "6":
    update_status()
```

```
elif choice == "7":
    print("Exit successfully")
    break
```

```
else:
    print("Invalid choice. Please enter a number from 1 to 7.")
```

```
main()
```

